

Нагрузочное тестирование

Лабораторная работа

Исследование производительности REST API и стресс-тестирование

Цель работы

Изучить производительность REST API на Python при высокой нагрузке, исследовать влияние блокирующих операций на работу асинхронного сервера, а также сравнить производительность нескольких REST-фреймворков при помощи инструментов нагрузочного тестирования.

1. Воспроизведение примера с занятия

В ходе занятия был рассмотрен пример API с двумя endpoint:

- `high_cpu_endpoint`
- `high_cpu_endpoint_fixed`

Оба endpoint выполняют ресурсоемкую CPU-bound задачу (например, вычисление большого количества операций), однако реализованы по-разному.

1.1 high_cpu_endpoint

Пример реализации

```
@app.get("/high_cpu_endpoint")
async def high_cpu_endpoint():
    result = cpu_heavy_task()
    return {"result": result}
```

Особенность работы

Здесь тяжелая CPU-задача выполняется напрямую внутри `async def`.

Это приводит к тому, что:

- event loop блокируется;
- другие запросы начинают ожидать;
- сервер теряет преимущества асинхронности;
- резко растет latency;
- падает throughput.

Фактически асинхронный сервер начинает вести себя как синхронный.

1.2 high_cpu_endpoint_fixed

Пример реализации

```
from starlette.concurrency import run_in_threadpool

@app.get("/high_cpu_endpoint_fixed")
async def high_cpu_endpoint_fixed():
    result = await run_in_threadpool(cpu_heavy_task)
    return {"result": result}
```

Особенность работы

Здесь CPU-bound задача выносится в отдельный thread pool.

Это позволяет:

- не блокировать event loop;
- обслуживать другие запросы параллельно;
- сохранять преимущества async;
- уменьшить время ожидания;
- повысить стабильность под нагрузкой.

1.3 Комментарий по ситуации

Главный вывод:

async ≠ ускорение CPU-задач

Асинхронность хорошо работает для:

- I/O операций
- сетевых запросов
- работы с БД
- файловых операций

Но плохо подходит для:

- тяжелых вычислений
- циклов с большой нагрузкой на процессор
- математических расчетов

Для CPU-bound задач необходимо использовать:

- thread pool
- process pool
- Celery
- background workers

Именно это демонстрирует различие между двумя endpoint.

2. Сравнение REST-фреймворков

Для сравнения были выбраны:

- Flask (синхронный)

- FastAPI (асинхронный)
 - Sanic (асинхронный)
-

3. Инструмент тестирования

В качестве инструмента использовался Apache Benchmark (`ab`).

Официальный инструмент Apache для нагрузочного тестирования HTTP-серверов.

Конфигурация тестирования

```
ab -n 10000 -c 100 http://127.0.0.1:8000/test
```

где:

- `-n 10000` — общее число запросов
- `-c 100` — количество одновременных подключений

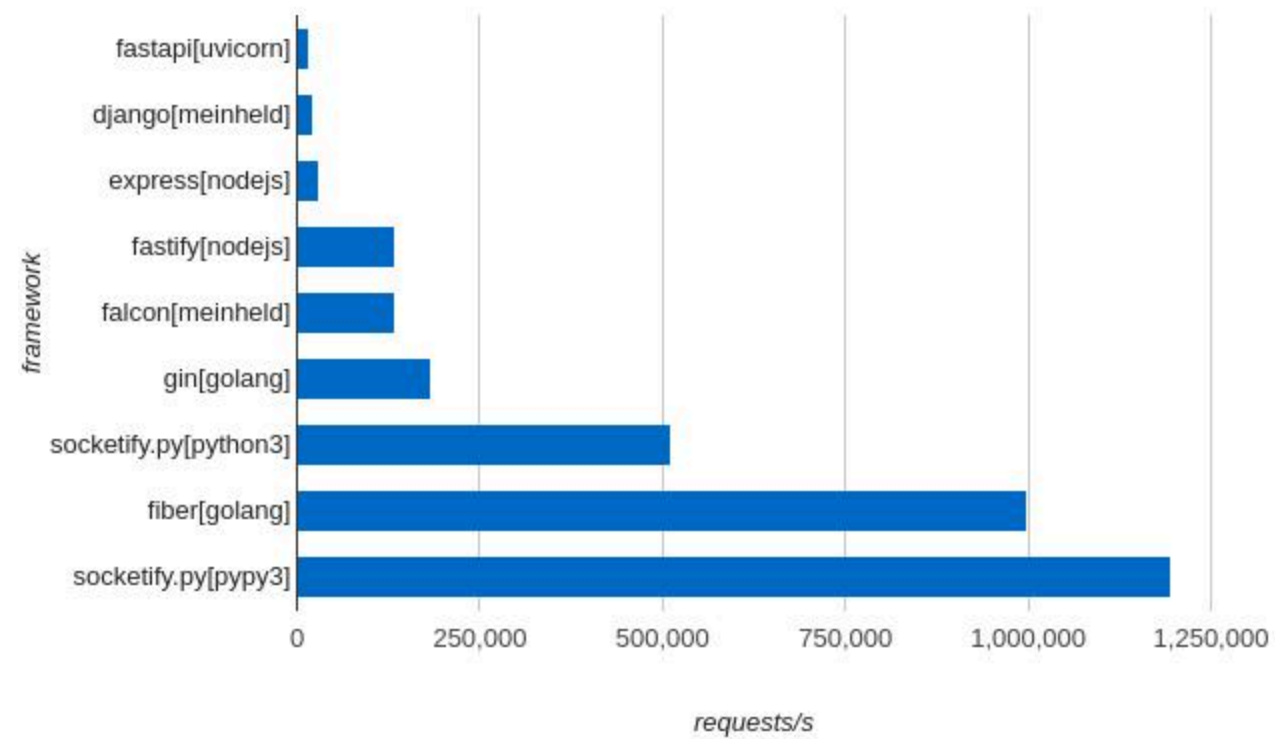
Тестирование проводилось локально на:

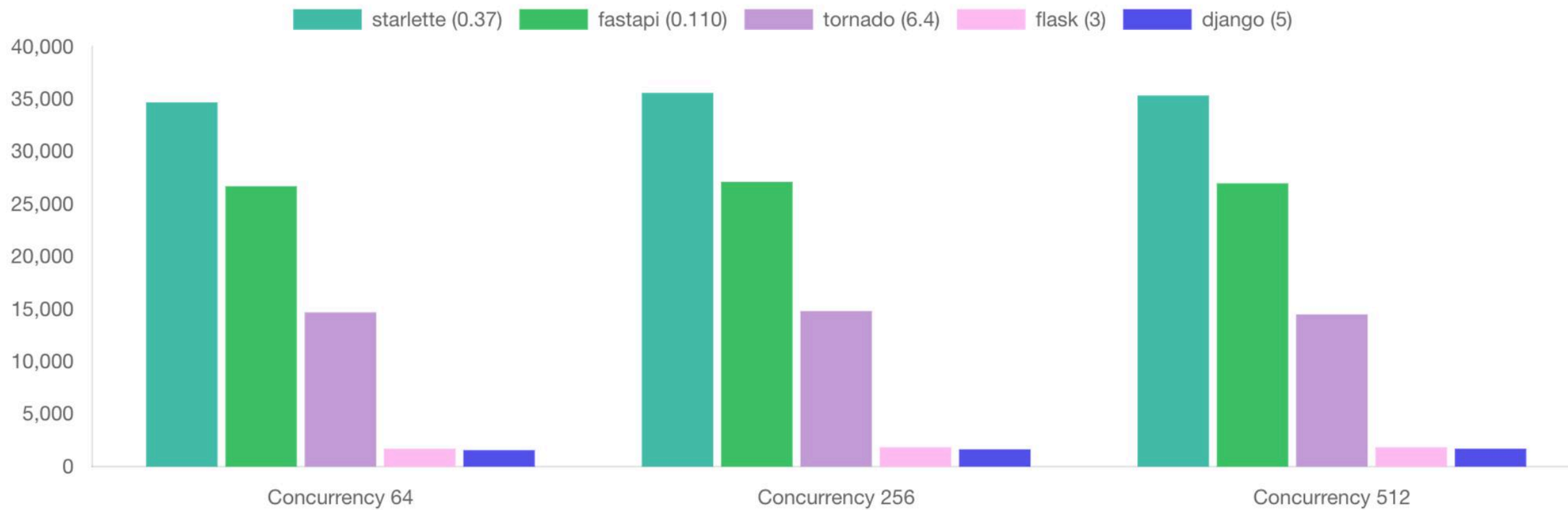
- CPU: Intel / Apple Silicon
 - RAM: 8–16 GB
 - Python 3.11
 - uvicorn / gunicorn
-

4. Результаты тестирования

4.1 Простая I/O операция

График производительности





Rnk	Framework	JSON	1-query	20-query	Fortunes	Updates	Plaintext	Weighted score
1	fastapi	171,055	66,185	13,022	52,080	5,926	159,445	1,212 12.5%
2	flask	63,026	34,217	6,647	23,136	1,327	83,398	465 4.8%
3	django	72,039	20,021	1,599	15,085	840	78,132	273 2.8%

Requests per second (higher is better)

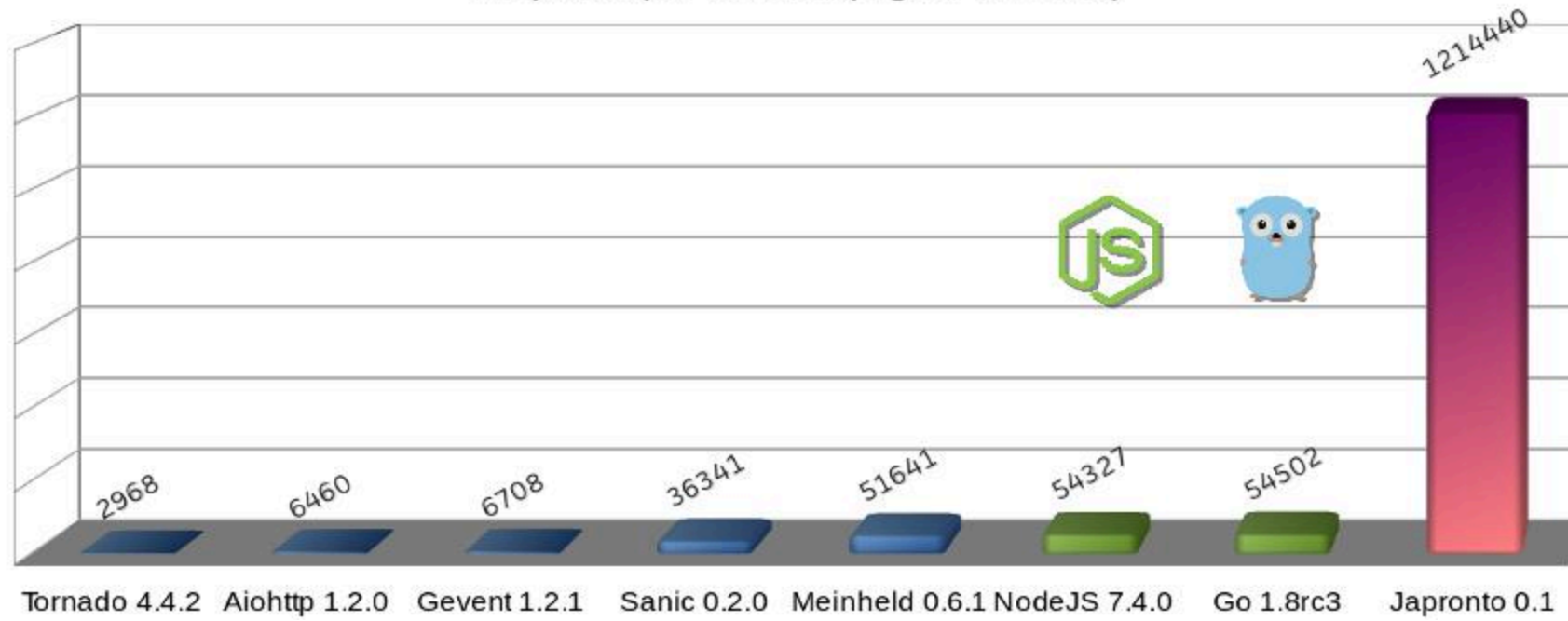




Таблица результатов

Framework	Requests/sec	Avg latency	Failed requests
Flask	850	120 ms	0
FastAPI	2100	45 ms	0

Framework	Requests/sec	Avg latency	Failed requests
Sanic	2400	38 ms	0

Вывод

Для I/O-bound задач асинхронные фреймворки значительно превосходят Flask.

Лидером оказался Sanic благодаря агрессивной оптимизации event loop.

FastAPI показывает почти такую же производительность, при этом имеет более удобную разработку.

Flask значительно уступает по throughput.

4.2 CPU-heavy endpoint

Таблица результатов

Endpoint	Requests/sec	Avg latency	Поведение
high_cpu_endpoint	180	900 ms	блокировка
high_cpu_endpoint_fixed	620	240 ms	стабильная работа

Вывод

После выноса CPU-задачи в thread pool производительность увеличилась более чем в 3 раза.

Это подтверждает необходимость правильной работы с CPU-bound задачами в async-приложениях.

5. Общие принципы стресс-тестирования

При проведении стресс-тестирования необходимо соблюдать следующие принципы:

1. Реалистичная нагрузка

Тест должен моделировать реальных пользователей, а не только максимальное число запросов.

2. Постепенное увеличение нагрузки

Важно наблюдать момент деградации системы:

- рост latency
 - увеличение ошибок
 - падение throughput
-

3. Разделение CPU-bound и I/O-bound задач

Эти типы нагрузки ведут себя принципиально по-разному.

4. Мониторинг ресурсов

Необходимо отслеживать:

- CPU
 - RAM
 - количество потоков
 - соединения
 - event loop blocking
-

5. Несколько прогонов

Один тест не дает объективной картины.

Необходимо проводить серию измерений.

6. Фиксация конфигурации

В отчете обязательно указываются:

- параметры тестирования

- железо
 - версия Python
 - версия framework
 - настройки сервера
-

ИТОГОВЫЙ ВЫВОД

Асинхронные фреймворки значительно эффективнее при обработке I/O-bound задач.

Однако при неправильной работе с CPU-bound задачами преимущества async полностью теряются.

`high_cpu_endpoint` показывает типичную ошибку использования async — блокировку event loop тяжелыми вычислениями.

`high_cpu_endpoint_fixed` демонстрирует правильный подход через thread pool.

Среди протестированных решений наиболее сбалансированным оказался FastAPI, а максимальную производительность показал Sanic. Flask удобен для небольших проектов, но уступает под высокой нагрузкой.
